重 庆 交 通 大 学

学生实验报告

实验课程名称：　《嵌入式系统》

实验项目名称：　STM32F103 的 uC/OS-II 移植与 AHT20 多任务采集

学　　院：　信息科学与工程学院

年级/专业班：　2024 级　通信 2403 班

学生姓名/学号：　徐正昕　　632407030612

指导教师：　娄路

2026 年 10 月

STM32F103 的 uC/OS-II 移植与 AHT20 多任务采集

—— 嵌入式实时操作系统与 I2C 传感器实验报告

一、实验目的

1. 理解嵌入式实时操作系统的基本组成：任务、任务控制块、就绪表、调度器、系统节拍和时间管理。
2. 掌握 uC/OS-II 在 ARM Cortex-M3（STM32F103）上的移植方法，理解 os_cpu.h、os_cpu_a.asm、os_cpu_c.c 三个移植文件的作用。
3. 掌握 SysTick、PendSV、BASEPRI 在 RTOS 移植中的作用，能够正确接入 HAL 库并解决 HAL 与 uC/OS-II 的时基冲突。
4. 构建 3 个应用任务：两个 LED 任务分别以 1 s 和 3 s 周期闪烁，第三个任务每 2 s 通过 I2C 读取 AHT20 温湿度，并转换为可读字符串从串口输出。
5. 理解软件 I2C 与硬件 I2C 的区别，掌握 AHT20 的地址、初始化命令、触发测量命令、7 字节数据格式和温湿度换算公式。
6. 记录从建立工程、复制源码、配置内核、修改中断、接入 I2C/AHT20 到编译通过的完整过程，并总结常见问题及解决方法。

二、实验环境与硬件资源

表 1　实验环境

项目	配置
MCU	STM32F103C8T6，Cortex-M3，64 KB Flash，20 KB SRAM，72 MHz
开发工具	Keil MDK 5.06，ARMCC V5.06 update 7（AC5）
RTOS	uC/OS-II V2.93.01（Apache-2.0 源码）
HAL 库	STM32Cube FW_F1 V1.8.5
串口工具	USB-TTL，115200-8-N-1，UTF-8 编码

表 2　硬件连接

信号	STM32 引脚	连接说明
红灯 LED1	PA0	PA0 → 220 Ω → LED → GND，高电平点亮
绿灯 LED2	PB0	PB0 → 220 Ω → LED → GND，高电平点亮
AHT20 SCL	PB6	I2C1_SCL，模块 VCC=3.3 V，GND 共地
AHT20 SDA	PB7	I2C1_SDA，模块自带或外接 4.7 kΩ 上拉
串口 TX	PA9	接 USB-TTL 的 RXD
串口 RX	PA10	接 USB-TTL 的 TXD
地	GND	与 USB-TTL 共地

系统时钟采用 HSE 8 MHz 经 PLL 9 倍频到 72 MHz；APB1 为 36 MHz，APB2 为 72 MHz。USART1 挂在 APB2 上，波特率 115200。LED 采用外部 LED，串联 220 Ω 限流电阻，高电平点亮。AHT20 与 MCU 共用 3.3 V 电源，SCL/SDA 分别接 PB6/PB7，I2C1 配置为 400 kHz 快速模式。

三、uC/OS-II 移植原理

3.1 uC/OS-II 的文件结构

uC/OS-II 的内核与处理器无关，移植时只需要提供处理器相关的端口层。工程中使用的文件可以分为三类：

表 3　uC/OS-II 工程文件分类

类别	主要文件	作用
内核	os_core.c、os_task.c、os_time.c	任务控制块、就绪表、调度、延时与节拍
可选服务	os_flag.c、os_mbox.c、os_mem.c、os_mutex.c、os_q.c、os_sem.c、os_tmr.c	事件标志、邮箱、内存、互斥量、队列、信号量、软件定时器
移植层	os_cpu.h、os_cpu_a.asm、os_cpu_c.c	数据类型、临界区、堆栈方向、上下文切换、SysTick 和钩子函数
配置	os_cfg.h、app_cfg.h、includes.h	内核裁剪、任务优先级、堆栈大小、公共头文件

本实验只使用任务管理、时间管理和调度器，关闭事件标志、邮箱、内存管理、互斥量、队列、信号量和软件定时器，以减小代码体积。

3.2 Cortex-M3 的上下文切换

Cortex-M3 有两个堆栈指针：MSP（主堆栈）和 PSP（进程堆栈）。uC/OS-II 让所有任务运行在 Thread 模式并使用 PSP，而异常服务程序使用 MSP。发生异常时，硬件自动把 R0～R3、R12、LR、PC、xPSR 压入当前堆栈；移植层的

PendSV 处理程序再补充保存 R4～R11 和 LR，并切换 PSP，从而完成任务上下文切换。

PendSV 被设置为最低优先级，因此它可以在所有其他中断处理完之后再执行，避免在中断嵌套过程中切换任务。OSCtxSw() 和 OSIntCtxSw() 只负责触发 PendSV；真正的切换在 OS_CPU_PendSVHandler 中完成。SysTick 则提供 1 ms 系统节拍，驱动 OSTimeTick() 和延时任务的就绪。

临界区使用 BASEPRI 实现。本实验把 CPU_CFG_KA_IPL_BOUNDARY 设为 4，因此优先级 0～3 的中断不受内核临界区屏蔽，优先级 4～15 的中断会被屏蔽。SysTick 和 USART1 的优先级都设为不低于 4，保证它们不会破坏内核数据结构。

3.3 移植层需要实现的接口

表 4　移植层接口

接口	位置	本实验实现
OS_CPU_SR_Save / Restore	os_cpu_a.asm	保存/恢复 BASEPRI，实现临界区
OSStartHighRdy	os_cpu_a.asm	设置 PSP、MSP、PendSV 优先级，启动第一个任务
OSCtxSw / OSIntCtxSw	os_cpu_a.asm	触发 PendSV
OS_CPU_PendSVHandler	os_cpu_a.asm	保存 R4～R11，切换任务堆栈
OS_CPU_SysTickHandler	os_cpu_c.c	OSIntEnter、OSTimeTick、OSIntExit
OS_CPU_SysTickInitFreq	os_cpu_c.c	按 72 MHz / 1000 Hz 配置 SysTick
OSInitHookBegin / OSTaskSwHook	os_cpu_c.c	异常栈初始化、任务切换钩子

3.4 软件 I2C 与硬件 I2C

I2C 是两根线的同步串行总线：SCL 为时钟，SDA 为数据，两者都是开漏结构，必须接上拉电阻。主机通过起始条件（SCL 高时 SDA 下降）、7 位地址 + 读写位、应答位、数据字节和停止条件（SCL 高时 SDA 上升）完成一次传输。AHT20 的 7 位地址是 0x38，HAL 库使用 8 位写地址 0x70。

表 5　软件 I2C 与硬件 I2C 对比

对比项	软件 I2C	硬件 I2C
实现方式	GPIO 翻转模拟 SCL/SDA 时序	使用 STM32 片内 I2C 外设
引脚	任意 GPIO，可灵活改线	固定复用引脚，STM32F103 常用 PB6/PB7
速率	通常 100 kHz 以下，受 CPU 和中断影响	可达 100/400 kHz，时序由硬件保证
CPU 占用	高，每个位都要软件延时和翻转	低，可用中断/DMA，CPU 只处理结果
可靠性	依赖延时，容易被中断打断	硬件状态机、错误标志完整，抗干扰更好
适用场景	引脚受限、时序特殊、教学演示	正式产品、多从机、高速或低功耗场景

本实验采用硬件 I2C1：PB6=SCL、PB7=SDA，400 kHz，7 位地址模式。硬件 I2C 的优点是时序由外设保证，CPU 不需要逐位翻转；缺点是对引脚复用、上拉电阻和总线状态要求更严格。若用软件 I2C，则只需把 PB6/PB7 配成开漏输出，用 GPIO 产生起始、停止、应答和字节时序，灵活但会占用更多 CPU 时间。

3.5 AHT20 传感器与数据格式

AHT20 上电后需要约 40 ms 稳定时间。主机先读取状态字节，若 bit3（CAL）为 0，则发送初始化命令 0xBE 0x08 0x00，等待 10 ms；然后发送触发测量命令 0xAC 0x33 0x00，等待约 80 ms，再读取 7 个字节：状态、湿度 20 位、温度 20 位和 CRC。

$$T(℃) = raw_t \times 200 / 2^{20} - 50$$

$$RH(\%) = raw_h \times 100 / 2^{20}$$

其中 raw_t 和 raw_h 都是 20 位无符号整数，raw_h 由 buf[1]～buf[3] 的高 20 位组成，raw_t 由 buf[3] 的低 4 位和 buf[4]、buf[5] 组成。AHT20 的测量范围约为 −40～85 ℃、0～100 %RH；本实验对结果做了限幅，防止异常数据导致显示或串口输出越界。

四、移植过程

4.1 建立基础工程

以已经调通的 STM32F10x_USART_Hello HAL 工程为基础，复制为 STM32F10x_uCOSII_RTOS。基础工程已经包含 HAL、CMSIS、启动文件、系统时钟和 USART1 初始化，因此可以集中精力完成 RTOS 移植。复制后删除原工程的编译产物和旧工程文件，只保留源码、Drivers、启动文件和 Keil 工程。

新建 Middlewares/uCOSII 目录，并在其中建立 Source、Ports、Cfg 三个子目录；再建立 App 目录存放应用任务。最终目录结构如下：

代码 1　工程目录结构

```
STM32F10x_uCOSII_RTOS/
├ Core/                        HAL 初始化、main.c、中断服务
├ Drivers/                     STM32F1 HAL 与 CMSIS
├ Middlewares/uCOSII/
│  ├ Source/                   uC/OS-II 内核源码
│  ├ Ports/                    os_cpu.h、os_cpu_a.asm、os_cpu_c.c、os_dbg.c
│  └ Cfg/                      os_cfg.h、app_cfg.h、includes.h
├ App/                         app.c、app.h
```

```
└ MDK-ARM/                    uCOSII_RTOS.uvprojx
```

4.2 复制 uC/OS-II 源码与 Cortex-M3 端口

从 uC/OS-II V2.93.01 源码包中复制 Source 目录下的 os_core.c、os_task.c、os_time.c 等内核文件，以及 ucos_ii.h、os_trace.h。注意不要同时把 ucos_ii.c 加入工程：ucos_ii.c 只是把各个 .c 文件再次包含进来的“合集文件”，和逐个加入源文件的方式二选一，否则会出现重复定义。

端口层选用 Ports/ARM-Cortex-M/ARMv7-M/ARM 目录，适用于 Cortex-M3 和 ARMCC（AC5）：

- os_cpu.h：数据类型、OS_CRITICAL_METHOD、OS_STK_GROWTH、OS_TASK_SW 等宏；
- os_cpu_a.asm：OSStartHighRdy、OSCtxSw、OSIntCtxSw、OS_CPU_PendSVHandler、OS_CPU_SR_Save/Restore；
- os_cpu_c.c：OSInitHookBegin、OSTaskSwHook、OS_CPU_SysTickHandler、OS_CPU_SysTickInitFreq 等；
- os_dbg.c：调试变量（本实验关闭 OS_DEBUG_EN，但仍加入工程以保持完整）。

STM32F103 是 Cortex-M3，没有硬件 FPU。官方端口默认根据 __TARGET_FPU_SOFTVFP 判断是否保存浮点寄存器；为避免 AC5 与端口宏不一致导致 OS_CPU_FP_Reg_Push/Pop 未定义，本实验在 os_cpu.h 中显式把 OS_CPU_ARM_FP_EN 设为 0。

4.3 Keil 工程组与头文件路径

在 Keil 工程中新增三个组：Middlewares/uCOSII/Source、Middlewares/uCOSII/Ports、App。Source 组加入除 ucos_ii.c 外的内核 .c 文件；Ports 组加入 os_cpu_c.c、os_cpu_a.asm、os_dbg.c；App 组加入 app.c。os_cpu_a.asm 的文件类型必须选择 Assembler Source（FileType=2），否则 Keil 会把它当成 C 文件编译。

在 C/C++ 的 Include Paths 中加入：

代码 2 Keil 头文件路径

```
../Core/Inc
../Drivers/STM32F1xx_HAL_Driver/Inc
```

```
../Drivers/STM32F1xx_HAL_Driver/Inc/Legacy
../Drivers/CMSIS/Device/ST/STM32F1xx/Include
../Drivers/CMSIS/Include
../Middlewares/uCOSII/Cfg
../Middlewares/uCOSII/Ports
../Middlewares/uCOSII/Source
../App
```

全局宏保持 USE_HAL_DRIVER,STM32F103xB。编译器选择 ARMCC V5.06 update 7，与官方 ARMv7-M/ARM 端口的汇编语法一致。

4.4 配置 os_cfg.h

os_cfg.h 决定内核裁剪。本实验只保留任务管理和时间管理，关键配置如下：

代码 3　os_cfg.h 关键配置

```
#define OS_MAX_TASKS             8u
#define OS_LOWEST_PRIO          15u
#define OS_TICKS_PER_SEC      1000u
#define OS_TASK_IDLE_STK_SIZE  128u
#define OS_TASK_STAT_EN          0u
#define OS_TASK_CREATE_EN        1u
#define OS_TASK_CREATE_EXT_EN    0u
#define OS_TASK_SW_HOOK_EN       1u
#define OS_TIME_DLY_HMSM_EN      1u
#define OS_TMR_EN                0u
#define OS_FLAG_EN               0u
#define OS_MBOX_EN               0u
#define OS_MEM_EN                0u
#define OS_MUTEX_EN              0u
#define OS_Q_EN                  0u
#define OS_SEM_EN                0u
```

OS_TICKS_PER_SEC = 1000 表示 1 ms 一个节拍，OSTimeDlyHMSM 的毫秒参数才能准确工作。OS_TASK_STAT_EN = 0 关闭统计任务，减少一个任务和一段代码。OS_TASK_SW_HOOK_EN 必须为 1，因为 os_cpu_a.asm 的 OSStartHighRdy 会调用 OSTaskSwHook；如果设为 0，链接时会报 Undefined symbol OSTaskSwHook。

4.5 配置 app_cfg.h

app_cfg.h 定义 CPU 优先级边界、任务优先级和堆栈大小。Cortex-M3 的 NVIC 有 4 位优先级，优先级数值越小越高。

代码 4　app_cfg.h 任务配置

```
#define CPU_CFG_KA_IPL_BOUNDARY   4u
#define CPU_CFG_NVIC_PRIO_BITS    4u


#define APP_TASK_LED1_PRIO        5u
#define APP_TASK_LED2_PRIO        6u
#define APP_TASK_UART_PRIO        7u


#define APP_TASK_LED1_STK_SIZE  256u
```

```
#define APP_TASK_LED2_STK_SIZE  256u
#define APP_TASK_UART_STK_SIZE  512u
```

BASEPRI 边界设为 4：优先级 0～3 为非内核感知中断，不会被 OS_ENTER_CRITICAL() 屏蔽；优先级 4～15 为内核感知中断，会被临界区屏蔽。SysTick 使用 4，USART1 使用 6，二者都在内核感知范围内。

4.6 includes.h 的包含顺序

ucos_ii.h 会包含 app_cfg.h、os_cfg.h 和 os_cpu.h。由于 os_cpu.h 在编译期检查 CPU_CFG_KA_IPL_BOUNDARY 和 CPU_CFG_NVIC_PRIO_BITS，app_cfg.h 必须先于 os_cpu.h 被包含。公共头文件如下：

代码 5 includes.h

```
#ifndef INCLUDES_H
#define INCLUDES_H
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include "stm32f1xx_hal.h"
#include "app_cfg.h"
#include "os_cpu.h"
#include "ucos_ii.h"
#endif
```

4.7 main.c 的初始化顺序

uC/OS-II 启动前不能有系统节拍中断进入内核，否则 OSTimeTick 可能在 TCB 尚未建立时访问空指针。因此初始化顺序非常关键：

代码 6 main.c 初始化顺序

```
HAL_Init();                  /* HAL 先把 SysTick 配成 1 ms */
__disable_irq();             /* 在 OSInit 完成前关闭全局中断 */

SystemClock_Config();        /* 72 MHz */
MX_GPIO_Init();              /* PA0、PB0 */
MX_USART1_UART_Init();       /* 115200-8-N-1 */

OSInit();                    /* 初始化内核 */
App_Init();                  /* 创建 3 个任务 */
OS_CPU_SysTickInitFreq(SystemCoreClock);  /* 72000 计数 -> 1 ms */
OSStart();                   /* 启动调度，开中断，不再返回 */
```

HAL_Init() 内部会配置并打开 SysTick 中断，此时 uC/OS-II 还没初始化；紧接着 __disable_irq() 把中断关掉。
OS_CPU_SysTickInitFreq(SystemCoreClock) 重新设置 SysTick 重装值和优先级，OSStart() 内部的 OSStartHighRdy 在最后执行 CPSIE I，才真正开中断并运行第一个任务。

4.8 中断向量与 HAL 时基

Cortex-M3 的 SysTick 和 PendSV 必须指向 uC/OS-II 的移植函数。stm32f1xx_it.c 中修改如下：

代码 7 SysTick 与 PendSV 接入

```
void SysTick_Handler(void)
{
    HAL_IncTick();              /* HAL 的 1 ms 计数 */
    OS_CPU_SysTickHandler();    /* uC/OS-II 的节拍 */
}

void PendSV_Handler(void)
{
    OS_CPU_PendSVHandler();     /* 真正的任务上下文切换 */
}

void SVC_Handler(void)
{
    /* uC/OS-II Cortex-M3 端口使用 PendSV，SVC 保持为空 */
}
```

HAL_IncTick() 必须保留，因为 HAL_UART_Transmit 的超时判断依赖 HAL_GetTick()。先累加 HAL 计数，再进入 uC/OS-II 的节拍处理；即使用户任务在 OSTimeTick 中被切换，HAL 的毫秒计数也已经更新。

USART1 中断优先级改为 6，避免使用 0 这种非内核感知优先级。本实验串口只用查询发送，不开启接收中断；保留 USART1_IRQHandler 是为了后续扩展。

4.9 临界区与上下文切换

os_cpu_a.asm 用 BASEPRI 实现临界区。OS_CPU_SR_Save 先关中断、读回旧 BASEPRI，再把新边界写入 BASEPRI，最后开中断；这样临界区只屏蔽优先级不低于 4 的中断，不会被高优先级中断打断内核链表操作。

OSStartHighRdy 的关键步骤如下：设置 PendSV 为最低优先级；把 PSP 清 0，表示第一次上下文切换；把 MSP 切换到 OS_CPU_ExceptStkBase；调用 OSTaskSwHook；把 OSRunning 置 1；从最高优先级任务的 TCB 取出堆栈指针装入 PSP；恢复 R4～R11、R0～R3、R12、LR、PC，最后 CPSIE I 开中断并跳转到任务入口。

OS_CPU_PendSVHandler 在异常入口保存当前任务的 R4～R11 和 LR，更新 OSTCBCur->OSTCBStkPtr；然后调用 OSTaskSwHook，切换 OSPrioCur/OSTCBCur 到 OSTCBHighRdy，恢复新任务的寄存器并异常返回。由于 PendSV 优先级最低，它不会打断其他中断，保证了切换的原子性。

4.10 编译与工程设置

工程使用 Objects 作为输出目录，勾选 Create HEX File。完整重建后输出：

代码 8　Keil 编译结果

```
Rebuild target 'uCOSII_RTOS'
compiling main.c...
compiling os_core.c...
compiling os_task.c...
compiling os_time.c...
assembling os_cpu_a.asm...
compiling os_cpu_c.c...
compiling app.c...
compiling aht20.c...
compiling stm32f1xx_hal_i2c.c...
linking...
Program Size: Code=13928  RO-data=568  RW-data=64  ZI-data=7224
FromELF: creating hex file...
".\Objects\uCOSII_RTOS.axf" - 0 Error(s), 0 Warning(s).
```

4.11 接入 I2C1 与 AHT20

AHT20 通过 I2C1 接入：PB6=SCL，PB7=SDA，400 kHz，7 位地址，HAL 使用 8 位写地址 0x70。I2C1 初始化如下：

代码 9　I2C1 初始化

```
hi2c1.Instance = I2C1;
hi2c1.Init.ClockSpeed = 400000;
hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
hi2c1.Init.OwnAddress1 = 0;
hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
hi2c1.Init.OwnAddress2 = 0;
hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
HAL_I2C_Init(&hi2c1);
```

同时要在 stm32f1xx_hal_conf.h 中打开 HAL_I2C_MODULE_ENABLED，并在 HAL_I2C_MspInit 中把 PB6/PB7 配成 GPIO_MODE_AF_OD。AHT20 模块通常自带 4.7 kΩ 上拉；若使用裸传感器，必须在 SCL/SDA 上外接上拉电阻，否则 HAL_I2C_IsDeviceReady 会一直返回 HAL_ERROR。

AHT20 驱动保留裸机版本的初始化、触发、CRC 校验和温湿度换算逻辑，但把 HAL_Delay 全部替换成 OSTimeDlyHMSM。这样 40 ms、10 ms、80 ms 的等待期间任务进入阻塞态，LED 任务仍能按时运行，不会因为传感器等待而掉帧。

五、多任务设计

5.1 任务划分

表 6　任务划分

任务	优先级	堆栈	周期	动作	外设
Task_LED1	5	256 OS_STK（1 KB）	1 s	PA0 亮 0.5 s、灭 0.5 s	GPIOA / PA0
Task_LED2	6	256 OS_STK（1 KB）	3 s	PB0 亮 1.5 s、灭 1.5 s	GPIOB / PB0
Task_AHT20	7	512 OS_STK（2 KB）	2 s	I2C 读取 AHT20，串口发送温湿度	I2C1 / USART1
Idle	15	128 OS_STK（512 B）	—	空闲任务	—

任务 1 和任务 2 的“周期”指完整亮灭周期：1 s 周期由 500 ms 亮 + 500 ms 灭组成；3 s 周期由 1500 ms 亮 + 1500 ms 灭组成。这样既满足 1 s/3 s 的周期要求，也能清楚看到点亮-熄灭两个状态。三个任务都调用 OSTimeDlyHMSM 主动让出 CPU，没有使用 HAL_Delay，因此不会阻塞其他任务。

5.2 应用任务代码

代码 10　App/app.c

```
/*
*********************************************************************************************************
***********
* 文件：app.c
* 功能：uC/OS-II 应用任务
*   任务 1：PA0 红灯，周期 1 s（亮 500 ms、灭 500 ms）
*   任务 2：PB0 绿灯，周期 3 s（亮 1500 ms、灭 1500 ms）
*   任务 3：每 2 s 通过 I2C1 读取 AHT20 温湿度，并从 USART1 发送
*
* 硬件：
*   PA0  -> 220Ω -> 红色 LED -> GND（高电平点亮）
*   PB0  -> 220Ω -> 绿色 LED -> GND（高电平点亮）
*   PB6  -> AHT20 SCL（I2C1，400 kHz）
*   PB7  -> AHT20 SDA
*   PA9  -> USB-TTL RXD
*   PA10 <- USB-TTL TXD
*   GND  <-> USB-TTL GND
*********************************************************************************************************
***********
*/
#include "app.h"
#include "main.h"
#include "aht20.h"


extern UART_HandleTypeDef huart1;


/* ---------------------------- 任务堆栈 ---------------------------- */
static OS_STK App_TaskLed1Stk[APP_TASK_LED1_STK_SIZE];
static OS_STK App_TaskLed2Stk[APP_TASK_LED2_STK_SIZE];
static OS_STK App_TaskAht20Stk[APP_TASK_UART_STK_SIZE];


/* 串口发送缓冲：AHT20 温湿度字符串 */
static char g_msg[128];


/* ---------------------------- 任务 1：1 s 周期红灯 ---------------------------- */
static void App_TaskLed1(void *p_arg)
```

```
{
    (void)p_arg;

    for (;;)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);   /* 点亮红灯 */
        OSTimeDlyHMSM(0u, 0u, 0u, 500u);                      /* 亮 500 ms */
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET); /* 熄灭红灯 */
        OSTimeDlyHMSM(0u, 0u, 0u, 500u);                      /* 灭 500 ms */
    }
}

/* ---------------------------- 任务 2：3 s 周期绿灯 ---------------------------- */
static void App_TaskLed2(void *p_arg)
{
    (void)p_arg;

    for (;;)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);   /* 点亮绿灯 */
        OSTimeDlyHMSM(0u, 0u, 1u, 500u);                      /* 亮 1500 ms */
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); /* 熄灭绿灯 */
        OSTimeDlyHMSM(0u, 0u, 1u, 500u);                      /* 灭 1500 ms */
    }
}

/* ---------------------------- 任务 3：2 s 周期 AHT20 采集 + 串口发送 -------------------------
--- */
static void App_TaskAht20(void *p_arg)
{
    float temperature = 0.0f;
    float humidity = 0.0f;
    int len;

    (void)p_arg;

    if (AHT20_Init() != 0u)
    {
        const char *err = "AHT20 init failed, check PB6/PB7 and 3.3V\r\n";
        (void)HAL_UART_Transmit(&huart1, (uint8_t *)err,
                                (uint16_t)strlen(err), 1000u);
    }

    for (;;)
    {
        if (AHT20_Read(&temperature, &humidity) == 0u)
        {
            len = snprintf(g_msg, sizeof(g_msg),
                           "hello uc/OS! AHT20: T=%.2f C, H=%.2f %%RH\r\n",
                           temperature, humidity);
        }
        else
        {
            len = snprintf(g_msg, sizeof(g_msg),
                           "hello uc/OS! AHT20 read error\r\n");
        }

        if (len > 0)
        {
            (void)HAL_UART_Transmit(&huart1, (uint8_t *)g_msg,
                                    (uint16_t)len, 1000u);
        }
        OSTimeDlyHMSM(0u, 0u, 2u, 0u);                        /* 2 s 周期 */
    }
}
```

```
/* ---------------------------- 创建任务 ---------------------------- */
void App_Init(void)
{
    INT8U err;


    err = OSTaskCreate(App_TaskLed1,
                       (void *)0,
                       &App_TaskLed1Stk[APP_TASK_LED1_STK_SIZE - 1u],
                       APP_TASK_LED1_PRIO);
    if (err != OS_ERR_NONE) { Error_Handler(); }


    err = OSTaskCreate(App_TaskLed2,
                       (void *)0,
                       &App_TaskLed2Stk[APP_TASK_LED2_STK_SIZE - 1u],
                       APP_TASK_LED2_PRIO);
    if (err != OS_ERR_NONE) { Error_Handler(); }


    err = OSTaskCreate(App_TaskAht20,
                       (void *)0,
                       &App_TaskAht20Stk[APP_TASK_UART_STK_SIZE - 1u],
                       APP_TASK_UART_PRIO);
    if (err != OS_ERR_NONE) { Error_Handler(); }
}
```

5.3 AHT20 驱动与串口数据格式

AHT20 驱动放在 App/aht20.c，对外提供 AHT20_Init、AHT20_Read、AHT20_DataValid 和 AHT20_CrcErrorCount。读取成功后，任务用 snprintf 把温度、湿度格式化为一行 ASCII 字符串：

代码 11　AHT20 采集与串口发送

```
if (AHT20_Read(&temperature, &humidity) == 0u) {
    len = snprintf(g_msg, sizeof(g_msg),
        "hello uc/OS! AHT20: T=%.2f C, H=%.2f %%RH\r\n",
        temperature, humidity);
} else {
    len = snprintf(g_msg, sizeof(g_msg),
        "hello uc/OS! AHT20 read error\r\n");
}
HAL_UART_Transmit(&huart1, (uint8_t *)g_msg, (uint16_t)len, 1000u);
OSTimeDlyHMSM(0u, 0u, 2u, 0u);
```

串口参数 115200-8-N-1。AHT20 的温度单位为 ℃，湿度单位为 %RH；正常输出示例为 hello uc/OS! AHT20: T=25.34 C, H=60.12 %RH。若传感器未接好或地址不对，任务会输出 AHT20 read error，便于现场排查。

代码 12　App/aht20.c

```
/*
 * aht20.c - AHT20 温湿度传感器驱动（uC/OS-II 阻塞式 I2C）
 *
 * 参考 AHT20 数据手册：
 *   上电等待 40 ms
 *   初始化命令 0xBE 0x08 0x00，等待 10 ms
 *   触发测量   0xAC 0x33 0x00，等待 80 ms
 *   读取 7 字节：状态 + 湿度 20 bit + 温度 20 bit + CRC
 *
 * 与裸机版本的区别：所有延时都改为 OSTimeDlyHMSM，等待期间主动让出 CPU，
 * 不会阻塞 LED 任务和空闲任务。
 */
#include "main.h"
```

```c
#include "includes.h"
#include "aht20.h"


#define AHT20_CMD_INIT      0xBEU
#define AHT20_CMD_TRIGGER   0xACU
#define AHT20_STATUS_BUSY   0x80U
#define AHT20_STATUS_CAL    0x08U


static uint8_t s_valid = 0;
static uint8_t s_ready = 0;
static uint8_t s_crc_errors = 0;


static void AHT20_DelayMs(uint32_t ms)
{
    OSTimeDlyHMSM(0u, 0u, 0u, (INT16U)ms);
}


static uint8_t AHT20_Crc8(const uint8_t *data, uint8_t len)
{
    uint8_t crc = 0xFFU;
    uint8_t i, bit;


    for (i = 0; i < len; i++)
    {
        crc ^= data[i];
        for (bit = 0; bit < 8; bit++)
        {
            if (crc & 0x80U)
            {
                crc = (uint8_t)((crc << 1) ^ 0x31U);
            }
            else
            {
                crc = (uint8_t)(crc << 1);
            }
        }
    }
    return crc;
}


uint8_t AHT20_Init(void)
{
    uint8_t status = 0;
    uint8_t cmd[3];


    s_ready = 0;
    s_valid = 0;


    if (HAL_I2C_IsDeviceReady(&hi2c1, AHT20_I2C_ADDR, 3, 100) != HAL_OK)
    {
        return 1;
    }


    AHT20_DelayMs(40u);


    if (HAL_I2C_Master_Receive(&hi2c1, AHT20_I2C_ADDR, &status, 1, 100) != HAL_OK)
    {
        return 1;
    }


    /* 状态位 bit3=1 表示已校准；否则发送初始化命令 */
    if ((status & AHT20_STATUS_CAL) == 0U)
    {
        cmd[0] = AHT20_CMD_INIT;
        cmd[1] = 0x08U;
        cmd[2] = 0x00U;
        if (HAL_I2C_Master_Transmit(&hi2c1, AHT20_I2C_ADDR, cmd, 3, 100) != HAL_OK)
        {
            return 1;
        }
```

```
        AHT20_DelayMs(10u);
    }


    if (HAL_I2C_Master_Receive(&hi2c1, AHT20_I2C_ADDR, &status, 1, 100) != HAL_OK)
    {
        return 1;
    }


    s_ready = 1;
    return ((status & AHT20_STATUS_CAL) != 0U) ? 0U : 1U;
}


uint8_t AHT20_Read(float *temperature, float *humidity)
{
    uint8_t cmd[3] = {AHT20_CMD_TRIGGER, 0x33U, 0x00U};
    uint8_t buf[7];
    uint32_t raw_h, raw_t;
    uint8_t retry;


    if (temperature == 0 || humidity == 0)
    {
        return 1;
    }


    if (!s_ready)
    {
        if (AHT20_Init() != 0U)
        {
            s_valid = 0;
            return 1;
        }
    }


    for (retry = 0; retry < 3U; retry++)
    {
        if (HAL_I2C_Master_Transmit(&hi2c1, AHT20_I2C_ADDR, cmd, 3, 100) != HAL_OK)
        {
            s_valid = 0;
            AHT20_DelayMs(10u);
            continue;
        }


        AHT20_DelayMs(80u);


        if (HAL_I2C_Master_Receive(&hi2c1, AHT20_I2C_ADDR, buf, 7, 100) != HAL_OK)
        {
            s_valid = 0;
            continue;
        }


        if (buf[0] & AHT20_STATUS_BUSY)
        {
            AHT20_DelayMs(20u);
            continue;
        }


        /* CRC 校验：若失败只记录，不丢弃数据（不同批次模块 CRC 实现有差异） */
        if (AHT20_Crc8(buf, 6) != buf[6])
        {
            s_crc_errors++;
        }


        raw_h = ((uint32_t)buf[1] << 16) | ((uint32_t)buf[2] << 8) | (uint32_t)buf[3];
        raw_h >>= 4;
        raw_t = (((uint32_t)(buf[3] & 0x0FU)) << 16)
              | ((uint32_t)buf[4] << 8)
              | (uint32_t)buf[5];


        *humidity = ((float)raw_h * 100.0f) / 1048576.0f;
```

```
        *temperature = ((float)raw_t * 200.0f) / 1048576.0f - 50.0f;


        if (*humidity < 0.0f)      { *humidity = 0.0f; }
        if (*humidity > 100.0f)    { *humidity = 100.0f; }
        if (*temperature < -40.0f) { *temperature = -40.0f; }
        if (*temperature > 85.0f)  { *temperature = 85.0f; }


        s_valid = 1;
        return 0;
    }


    s_valid = 0;
    return 1;
}


uint8_t AHT20_DataValid(void)
{
    return s_valid;
}


uint8_t AHT20_CrcErrorCount(void)
{
    return s_crc_errors;
}
```

5.4 任务时序

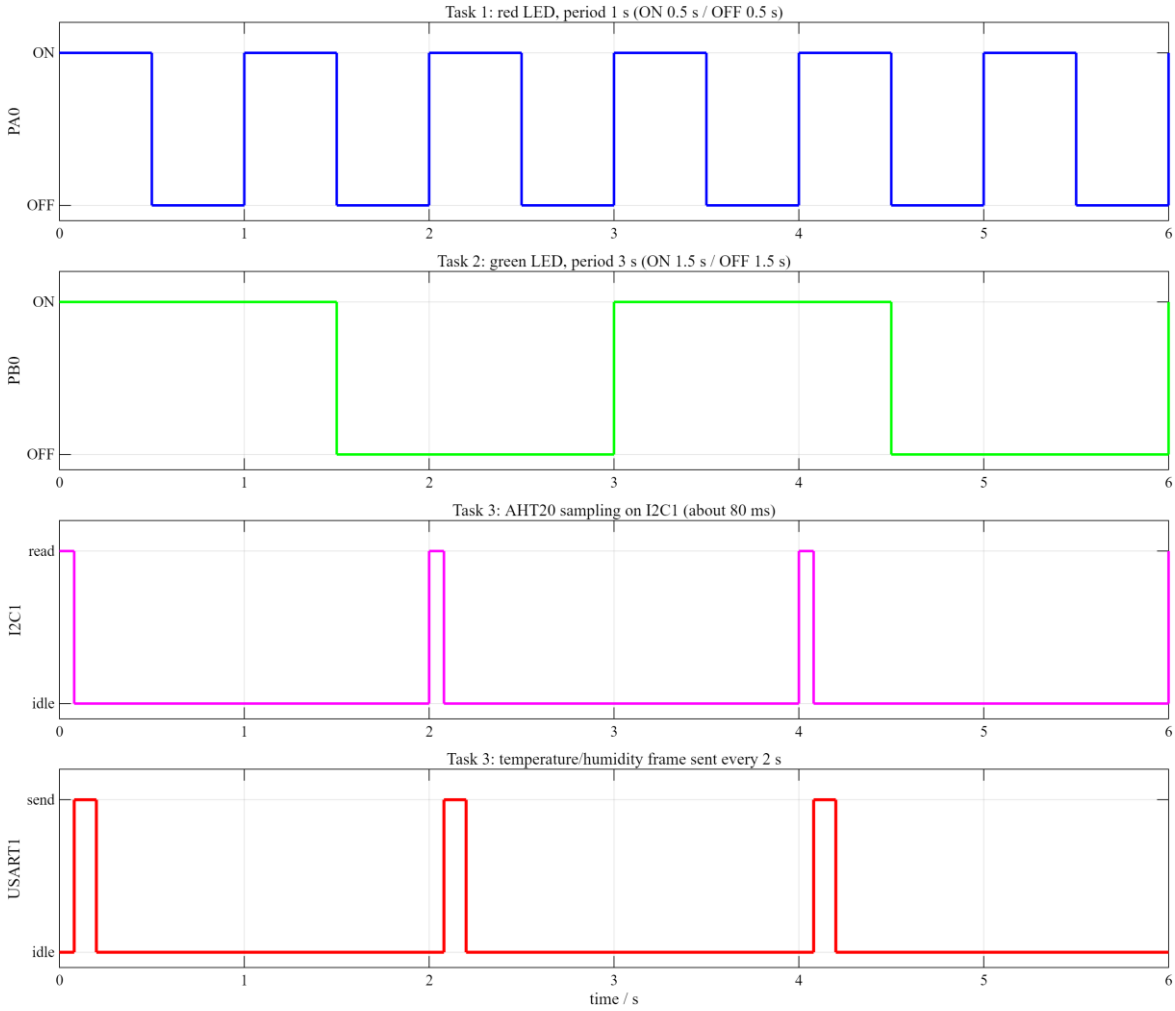


图 1　三个任务的预期时序（0～6 s）

图 1 给出 0～6 s 的预期时序：PA0 每 1 s 完成一次亮灭，PB0 每 3 s 完成一次亮灭，Task_AHT20 在 0、2、4、6 s 先通过 I2C1 读取 AHT20（约 80 ms），再把温湿度字符串从 USART1 发出。三个任务相互独立，LED 的闪烁不会因为传感器等待或串口发送而停止。

六、编译与运行结果

6.1 编译结果

表 7　编译与链接结果

项目	结果
目标器件	STM32F103C8，Cortex-M3
编译器	ARMCC V5.06 update 7（build 960）
Code	13928 字节
RO-data	568 字节
RW-data	64 字节
ZI-data	7224 字节
HEX 文件	41001 字节
错误 / 警告	0 / 0

ZI-data 约 7.1 KB，包含三个任务堆栈、空闲任务堆栈、异常栈、内核全局变量以及 AHT20 驱动的少量静态变量；STM32F103C8 有 20 KB SRAM，余量充足。Code 约 13.9 KB，其中 HAL I2C 和浮点格式化占比较大，但仍远小于 64 KB Flash。

6.2 预期运行现象

- 上电后 PA0 红灯以 1 s 为周期闪烁：亮 0.5 s、灭 0.5 s。
- PB0 绿灯以 3 s 为周期闪烁：亮 1.5 s、灭 1.5 s。
- USART1 每 2 s 输出一行 AHT20 温湿度，例如 hello uc/OS! AHT20: T=25.34 C, H=60.12 %RH。
- 若 AHT20 未接好或地址不对，串口输出 AHT20 read error，LED 任务仍正常运行。
- 三个任务互不阻塞；即使传感器等待 80 ms 或串口正在发送，LED 也不会停止闪烁。

本机当前未识别到 ST-Link 和 USB-TTL 串口设备，因此本次提交完成的是工程移植、编译与链接验证；实机现象需在实验台上烧录后确认。烧录与验证步骤如下：

1. 用 ST-Link 连接 SWD（SWDIO、SWCLK、GND、3.3 V），在 Keil 中选择 ST-Link Debugger，下载 Objects/uCOSII_RTOS.hex。
2. PA0、PB0 分别接红色、绿色 LED（串联 220 Ω 到 GND）；AHT20 的 VCC 接 3.3 V，GND 共地，SCL 接 PB6，SDA 接 PB7，并确认上拉电阻。
3. PA9 接 USB-TTL RXD，PA10 接 TXD，GND 共地；串口助手选择 115200-8-N-1。
4. 观察 AHT20 数值每 2 s 更新一次；用示波器测 PA0、PB0，周期应分别为 1 s 和 3 s。

6.3 关键代码分析

任务 1 和任务 2 的结构完全相同，只是端口、周期不同。调用 HAL_GPIO_WritePin 改变引脚电平后，立即调用 OSTimeDlyHMSM 进入阻塞态；调度器会选择下一个最高优先级就绪任务。由于 LED 任务优先级高于 AHT20 任务，LED 的亮灭时刻不会受到传感器读取或串口发送的影响。

任务 3 先调用 AHT20_Read，内部通过 HAL_I2C_Master_Transmit/Receive 完成触发和读取；等待 80 ms 时使用 OSTimeDlyHMSM，任务进入阻塞态，CPU 可以运行 LED 任务或空闲任务。读取成功后用 snprintf 格式化，再用 HAL_UART_Transmit 查询发送；发送约 5 ms，随后 OSTimeDlyHMSM(0,0,2,0) 进入 2 s 阻塞。若以后需要更严格的实时性，可以把串口发送改成中断或 DMA，并在发送完成回调中释放信号量。

App_Init 在 OSStart 之前创建三个任务，任务堆栈使用静态数组，避免动态内存分配。堆栈大小按经验留足余量：LED 任务 1 KB，AHT20/串口任务 2 KB，空闲任务 512 B。实际项目可以用 OSTaskStkChk 检查剩余堆栈，再进一步裁剪。

七、移植中的问题与解决

表 8　移植中的典型问题

现象	原因	解决方法
Keil 提示 Cannot read project file	uvprojx 被保存成带 BOM 的 UTF-8	用无 BOM 的 UTF-8 重新保存工程文件
找不到 ucos_ii.h	未加入 Cfg/Source/Ports/App 头文件路径	在 Include Paths 中补齐 4 个目录
找不到 os_core.c 等源文件	Source 目录为空或路径写错	重新复制源码，确认组内 FilePath 为 ../Middlewares/uCOSII/Source/...
L6218E: Undefined symbol OSTaskSwHook	OS_TASK_SW_HOOK_EN = 0，但汇编里调用了该钩子	把 OS_TASK_SW_HOOK_EN 设为 1
OS_CPU_FP_Reg_Push/Pop 未定义	Cortex-M3 无 FPU，端口宏与 AC5 的 FPU 设置不一致	在 os_cpu.h 中强制 OS_CPU_ARM_FP_EN = 0
程序跑飞或进 HardFault	SysTick 在 OSInit 前进入内核，或 PendSV 未接入	OSInit 前 __disable_irq，PendSV_Handler 调 OS_CPU_PendSVHandler
串口发送超时不返回	HAL_GetTick 没有递增	SysTick_Handler 中先调用 HAL_IncTick()
串口中文乱码	源文件编码与串口助手编码不一致	中文用 UTF-8 字节转义，串口助手选 UTF-8
OSTimeDlyHMSM 未定义	OS_TIME_DLY_HMSM_EN = 0	把 OS_TIME_DLY_HMSM_EN 设为 1
Error_Handler 隐式声明警告	app.c 未包含 main.h	在 app.c 中包含 main.h
HAL_I2C_Init 链接失败	stm32f1xx_hal_conf.h 未打开 HAL_I2C_MODULE_ENABLED	取消该宏的注释并加入 stm32f1xx_hal_i2c.c
AHT20 一直返回 HAL_ERROR	地址写成 0x38、PB6/PB7 未接上拉或传感器未供电	HAL 使用 8 位地址 0x70；检查 3.3 V 和 4.7 kΩ 上拉
AHT20 数据全 0 或 BUSY 一直为 1	触发后等待不足 80 ms，或初始化未完成	保证 40 ms/10 ms/80 ms 延时，读取状态 bit7

以上问题中，最容易忽略的是 SysTick 与 HAL 的冲突。HAL_Init 会先打开 SysTick，如果直接调用 OSInit/OSStart，SysTick 可能在 TCB 还未建立时进入 OSTimeTick，导致 HardFault。本实验用 __disable_irq() 把这段时间的中断关掉，等 OS_CPU_SysTickInitFreq 重新配置后再由 OSStart 开中断，问题彻底解决。

八、实验总结

本次实验完成了 uC/OS-II 在 STM32F103C8T6 上的移植，并把 I2C/AHT20 采集接入第三个任务。移植的核心不是“把源码复制进去”，而是把 Cortex-M3 的硬件机制和内核的调度需求对应起来：SysTick 提供 1 ms 节拍，PendSV 完成上下文切换，BASEPRI 实现临界区，PSP/MSP 分离任务栈和异常栈。只要这四件事配置正确，uC/OS-II 就可以稳定运行。

在工程配置上，最容易出错的是文件分组、头文件路径和条件编译宏。OS_TASK_SW_HOOK_EN、OS_TIME_DLY_HMSM_EN、OS_CPU_ARM_FP_EN 这三个宏分别影响链接、延时函数和浮点上下文，必须根据实际芯片和端口设置。接入 AHT20 时还要同时打开 HAL_I2C_MODULE_ENABLED、把 PB6/PB7 配成 AF_OD，并注意 HAL 使用 8 位地址 0x70。

通过三个任务的时序设计，我对“优先级 + 延时阻塞”的调度方式有了直观认识：LED 任务优先级高、周期短，AHT20 任务优先级低、周期长；每个任务都在延时期间让出 CPU，因此看起来是“同时运行”的。AHT20 的 80 ms 转换等待也被改造成 OSTimeDlyHMSM，进一步验证了 RTOS 中“阻塞等待”比 HAL_Delay 更适合多任务环境。

软件 I2C 与硬件 I2C 的对比让我理解了“灵活性与实时性”的取舍：软件 I2C 可以任意改引脚、便于教学和排查，但逐位翻转会占用 CPU；硬件 I2C 时序稳定、CPU 占用低，适合正式产品。本实验使用硬件 I2C1，把 PB6/PB7 留给 AHT20，USART1 留给上位机，资源划分清晰。

参考文献

[1] Labrosse J J. MicroC/OS-II: The Real-Time Kernel[M]. 2nd ed. CMP Books, 2002.

[2] Silicon Laboratories. uC/OS-II V2.93.01 Source Code and ARMv7-M Port[EB/OL]. https://github.com/weston-embedded/uC-OS2.

[3] STMicroelectronics. RM0008 Reference Manual: STM32F101xx/STM32F102xx/STM32F103xx[Z].

[4] ARM. Cortex-M3 Technical Reference Manual[Z].

[5] Keil. MDK-ARM Compiler User Guide[Z].

[6] 广州奥松电子. AHT20 温湿度传感器数据手册[Z].